

DEVARA HEPSIBA

2403A51L25

BATCH-51

Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Task 1: Merge Sort Implementation

Prompt: Generate a Python function `merge_sort(arr)` that sorts a list in ascending order using the Merge Sort algorithm. Include proper docstring explaining: Best case time complexity Average case time complexity Worst case time complexity Space complexity Also include a simple test case.

Code:

```
def merge_sort(arr):
    """
    Sorts a list in ascending order using the Merge Sort algorithm.

    Time Complexity:
    - Best case:  $O(n \log n)$ 
    - Average case:  $O(n \log n)$ 
    - Worst case:  $O(n \log n)$ 

    Space Complexity:  $O(n)$ 

    Parameters:
    arr (list): The list to be sorted.

    Returns:
    list: A new sorted list.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)

def merge(left, right):
    sorted_list = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if left[left_index] < right[right_index]:
            sorted_list.append(left[left_index])
            left_index += 1
        else:
            sorted_list.append(right[right_index])
            right_index += 1

    sorted_list.extend(left[left_index:])
    sorted_list.extend(right[right_index:])

    return sorted_list

# Test case
if __name__ == "__main__":
    test_array = [38, 27, 43, 3, 9, 82, 10]
    sorted_array = merge_sort(test_array)
    print("Sorted array:", sorted_array)
```

Output:

```
Sorted array: [3, 9, 10, 27, 38, 43, 82]
```

Explanation: Merge Sort is a divide-and-conquer algorithm that divides the list into halves and merges them after sorting.

It has a time complexity of $O(n \log n)$ in all cases.

It is stable and works well for large datasets.

However, it requires extra space for merging.

Task 2: Binary Search

Prompt: Create a Python function `binary_search(arr, target)` that performs binary search on a sorted list and returns the index of the target element or -1 if not found. Add docstring explaining best, average, and worst case time complexity and space complexity. Include example test cases.

Code:

```
def binary_search(arr, target):
    """
    Performs binary search on a sorted list.

    Time Complexity:
    - Best case: O(1)
    - Average case: O(log n)
    - Worst case: O(log n)

    Space Complexity: O(1)

    Parameters:
    arr (list): The sorted list to search.
    target: The element to search for.

    Returns:
    int: The index of the target element if found, otherwise -1.
    """
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1

# Example test cases
if __name__ == "__main__":
    test_array = [3, 9, 10, 27, 38, 43, 82]

    # Test case 1: Target exists in the array
    result = binary_search(test_array, 27)
    print(f"Index of element 27: {result}") # Expected output: Index of element 27: 3

    # Test case 2: Target does not exist in the array
    result = binary_search(test_array, 5)
    print(f"Index of element 5: {result}") # Expected output: Index of element 5: -1
```

Output:

```
Index of element 27: 3
Index of element 5: -1
```

Explanation: Binary Search works only on sorted data and repeatedly divides the search space into halves.

It has **$O(\log n)$** time complexity in average and worst cases.

It is much faster than linear search for large datasets.

It is efficient for searching unique IDs or sorted lists.

Task 3: Smart Healthcare Appointment Scheduling System

Prompt: A healthcare system stores appointment records as an unsorted list of tuples (appointment ID, patient name, doctor name, time, fee). Recommend suitable searching and sorting algorithms, justify the choices, and implement Python code to search by appointment ID and sort by time or consultation fee, printing the sorted output.

Code:

```
appointments = [
    (1, "John Doe", "Dr. Smith", "10:00 AM", 100),
    (2, "Jane Doe", "Dr. Brown", "11:00 AM", 150),
    (3, "Alice Smith", "Dr. Johnson", "9:00 AM", 120),
    (4, "Bob Johnson", "Dr. Lee", "1:00 PM", 80),
    (5, "Charlie Brown", "Dr. Davis", "2:00 PM", 200)
]

def search_appointment(appointment_id):
    for appointment in appointments:
        if appointment[0] == appointment_id:
            return appointment
    return "Appointment ID not found"

def sort_appointments_by_time():
    return sorted(appointments, key=lambda x: x[3])

def sort_appointments_by_fee():
    return sorted(appointments, key=lambda x: x[4])

# Example usage
print(search_appointment(3))
print("Appointments sorted by time:")
for appointment in sort_appointments_by_time():
    print(appointment)
print("Appointments sorted by consultation fee:")
for appointment in sort_appointments_by_fee():
    print(appointment)
```

Output:

```
(3, 'Alice Smith', 'Dr. Johnson', '9:00 AM', 120)
Appointments sorted by time:
(1, 'John Doe', 'Dr. Smith', '10:00 AM', 100)
(2, 'Jane Doe', 'Dr. Brown', '11:00 AM', 150)
(4, 'Bob Johnson', 'Dr. Lee', '1:00 PM', 80)
(5, 'Charlie Brown', 'Dr. Davis', '2:00 PM', 200)
(3, 'Alice Smith', 'Dr. Johnson', '9:00 AM', 120)
Appointments sorted by consultation fee:
(4, 'Bob Johnson', 'Dr. Lee', '1:00 PM', 80)
(1, 'John Doe', 'Dr. Smith', '10:00 AM', 100)
(3, 'Alice Smith', 'Dr. Johnson', '9:00 AM', 120)
(2, 'Jane Doe', 'Dr. Brown', '11:00 AM', 150)
(5, 'Charlie Brown', 'Dr. Davis', '2:00 PM', 200)
```

Explanation: Binary Search is suitable for searching appointment ID if records are sorted.

Sorting by time or fee can be done using efficient algorithms like Merge Sort or Python's built-in sort (TimSort).

Binary Search reduces search time significantly.

Efficient sorting helps in better appointment management and scheduling.

Task 4: Railway Ticket Reservation System

Prompt: Create a Python program for a Railway Ticket Reservation System. Each booking contains ticket ID, passenger name, train number, seat number, and travel date. Recommend efficient searching and sorting algorithms. Justify the algorithm choices. Implement: Search by ticket ID Sort bookings by travel date Sort bookings by seat number Include sample data and proper comments.

Code:

```
bookings = [
    (101, "Alice Smith", "Train A", "12A", "2024-07-01"),
    (102, "Bob Johnson", "Train B", "15B", "2024-07-03"),
    (103, "Charlie Brown", "Train A", "10C", "2024-07-02"),
    (104, "David Lee", "Train C", "20D", "2024-07-01"),
    (105, "Eve Davis", "Train B", "5E", "2024-07-04")
]

def search_booking(ticket_id):
    for booking in bookings:
        if booking[0] == ticket_id:
            return booking
    return "Ticket ID not found"

def sort_bookings_by_date():
    return sorted(bookings, key=lambda x: x[4])

def sort_bookings_by_seat():
    return sorted(bookings, key=lambda x: x[3])

# Example usage
print(search_booking(103))
print("Bookings sorted by travel date:")
for booking in sort_bookings_by_date():
    print(booking)
print("Bookings sorted by seat number:")
for booking in sort_bookings_by_seat():
    print(booking)
```

Output:

```
(103, 'Charlie Brown', 'Train A', '10C', '2024-07-02')
Bookings sorted by travel date:
(101, 'Alice Smith', 'Train A', '12A', '2024-07-01')
(104, 'David Lee', 'Train C', '20D', '2024-07-01')
(103, 'Charlie Brown', 'Train A', '10C', '2024-07-02')
(102, 'Bob Johnson', 'Train B', '15B', '2024-07-03')
(105, 'Eve Davis', 'Train B', '5E', '2024-07-04')
Bookings sorted by seat number:
(103, 'Charlie Brown', 'Train A', '10C', '2024-07-02')
(101, 'Alice Smith', 'Train A', '12A', '2024-07-01')
(102, 'Bob Johnson', 'Train B', '15B', '2024-07-03')
(104, 'David Lee', 'Train C', '20D', '2024-07-01')
(105, 'Eve Davis', 'Train B', '5E', '2024-07-04')
```

Explanation: Searching ticket ID can be done using Binary Search (if sorted) or Dictionary for faster lookup.

Sorting by travel date or seat number improves booking management.

Efficient sorting ensures organized reservation records.

Optimized algorithms improve system performance for large datasets.

Task 5: Smart Hostel Room Allocation System

Prompt: Develop a Python program for a Smart Hostel Room Allocation System. Each record contains student ID, room number, floor, and allocation date. Suggest optimized searching and sorting algorithms. Justify your selections. Implement: Search by student ID Sort by room number Sort by allocation date Add test cases and documentation.

Code:

```
allocations = [
    (201, "Room 101", "1st Floor", "2024-08-01"),
    (202, "Room 102", "1st Floor", "2024-08-03"),
    (203, "Room 201", "2nd Floor", "2024-08-02"),
    (204, "Room 202", "2nd Floor", "2024-08-01"),
    (205, "Room 301", "3rd Floor", "2024-08-04")
]

def search_allocation(student_id):
    for allocation in allocations:
        if allocation[0] == student_id:
            return allocation
    return "Student ID not found"

def sort_allocations_by_room():
    return sorted(allocations, key=lambda x: x[1])

def sort_allocations_by_date():
    return sorted(allocations, key=lambda x: x[3])

# Example usage
print(search_allocation(203))
print("Allocations sorted by room number:")
for allocation in sort_allocations_by_room():
    print(allocation)
print("Allocations sorted by allocation date:")
for allocation in sort_allocations_by_date():
    print(allocation)
```

Output:

```
(203, 'Room 201', '2nd Floor', '2024-08-02')
Allocations sorted by room number:
(201, 'Room 101', '1st Floor', '2024-08-01')
(202, 'Room 102', '1st Floor', '2024-08-03')
(203, 'Room 201', '2nd Floor', '2024-08-02')
(204, 'Room 202', '2nd Floor', '2024-08-01')
(205, 'Room 301', '3rd Floor', '2024-08-04')
Allocations sorted by allocation date:
(201, 'Room 101', '1st Floor', '2024-08-01')
(204, 'Room 202', '2nd Floor', '2024-08-01')
(203, 'Room 201', '2nd Floor', '2024-08-02')
(202, 'Room 102', '1st Floor', '2024-08-03')
(205, 'Room 301', '3rd Floor', '2024-08-04')
PS C:\Users\hepsi\OneDrive\Desktop\AI>
```

Explanation: Searching student ID is efficient using Binary Search or Hashing.
Sorting by room number or allocation date ensures structured record management.
Efficient algorithms reduce processing time.
This improves hostel administration efficiency.

Task 6: Online Movie Streaming Platform

Prompt: Create a Python program for an Online Movie Streaming Platform. Each movie record contains movie ID, title, genre, rating, and release year. Recommend suitable searching and sorting algorithms. Justify why they are efficient. Implement: Search by movie ID Sort by rating Sort by release year Include sample movie data and comments.

Code:

```
movies = [
    (301, "Inception", "Sci-Fi", 8.8, 2010),
    (302, "The Godfather", "Crime", 9.2, 1972),
    (303, "The Dark Knight", "Action", 9.0, 2008),
    (304, "Pulp Fiction", "Crime", 8.9, 1994),
    (305, "The Shawshank Redemption", "Drama", 9.3, 1994)
]

def search_movie(movie_id):
    for movie in movies:
        if movie[0] == movie_id:
            return movie
    return "Movie ID not found"

def sort_movies_by_rating():
    return sorted(movies, key=lambda x: x[3], reverse=True)

def sort_movies_by_release_year():
    return sorted(movies, key=lambda x: x[4])

# Example usage
print(search_movie(303))
print("Movies sorted by rating:")
for movie in sort_movies_by_rating():
    print(movie)
print("Movies sorted by release year:")
for movie in sort_movies_by_release_year():
    print(movie)
```

Output:

```
(303, 'The Dark Knight', 'Action', 9.0, 2008)
Movies sorted by rating:
(305, 'The Shawshank Redemption', 'Drama', 9.3, 1994)
(302, 'The Godfather', 'Crime', 9.2, 1972)
(303, 'The Dark Knight', 'Action', 9.0, 2008)
(304, 'Pulp Fiction', 'Crime', 8.9, 1994)
(301, 'Inception', 'Sci-Fi', 8.8, 2010)
Movies sorted by release year:
(302, 'The Godfather', 'Crime', 9.2, 1972)
(304, 'Pulp Fiction', 'Crime', 8.9, 1994)
(305, 'The Shawshank Redemption', 'Drama', 9.3, 1994)
(303, 'The Dark Knight', 'Action', 9.0, 2008)
(301, 'Inception', 'Sci-Fi', 8.8, 2010)
```

Explanation: Binary Search helps quickly locate a movie by ID.
Sorting by rating or release year helps users browse better content.
Efficient sorting algorithms improve performance.
Optimized searching enhances user experience.

Task 7: Smart Agriculture Crop Monitoring System

Prompt: Design a Python program for a Smart Agriculture Crop Monitoring System. Each crop record contains crop ID, crop name, soil moisture level, temperature, and yield estimate. Use AI reasoning to select appropriate searching and sorting algorithms. Justify algorithm suitability. Implement: Search by crop ID Sort by soil moisture level Sort by yield estimate Provide sample data and explanation.

Code:

```
crops = [
    (401, "Wheat", 30, 25, 1000),
    (402, "Corn", 40, 30, 1500),
    (403, "Rice", 50, 28, 2000),
    (404, "Barley", 20, 22, 800),
    (405, "Soybean", 35, 27, 1200)
]

def search_crop(crop_id):
    for crop in crops:
        if crop[0] == crop_id:
            return crop
    return "Crop ID not found"

def sort_crops_by_moisture():
    return sorted(crops, key=lambda x: x[2])

def sort_crops_by_yield():
    return sorted(crops, key=lambda x: x[4], reverse=True)

# Example usage
print(search_crop(403))
print("Crops sorted by soil moisture level:")
for crop in sort_crops_by_moisture():
    print(crop)
print("Crops sorted by yield estimate:")
for crop in sort_crops_by_yield():
    print(crop)
```

Output:

```
(403, 'Rice', 50, 28, 2000)
Crops sorted by soil moisture level:
(404, 'Barley', 20, 22, 800)
(401, 'Wheat', 30, 25, 1000)
(405, 'Soybean', 35, 27, 1200)
(402, 'Corn', 40, 30, 1500)
(403, 'Rice', 50, 28, 2000)
Crops sorted by yield estimate:
(403, 'Rice', 50, 28, 2000)
(402, 'Corn', 40, 30, 1500)
(405, 'Soybean', 35, 27, 1200)
(401, 'Wheat', 30, 25, 1000)
(404, 'Barley', 20, 22, 800)
```

Explanation: Searching crop ID is efficient using Binary Search or Hash Map.
Sorting by moisture level or yield estimate helps farmers analyze crops easily.
Efficient algorithms reduce computation time.
This supports better agricultural decision-making.

Task 8: Airport Flight Management System

Prompt: Develop a Python program for an Airport Flight Management System. Each flight record contains flight ID, airline name, departure time, arrival time, and status. Recommend appropriate searching and sorting algorithms. Justify your algorithm selection. Implement: Search by flight ID Sort by departure time Sort by arrival time Include sample test data and proper documentation.

Code:

```
flights = [
    (501, "Airline A", "08:00 AM", "10:00 AM", "On Time"),
    (502, "Airline B", "09:00 AM", "11:00 AM", "Delayed"),
    (503, "Airline C", "07:30 AM", "09:30 AM", "On Time"),
    (504, "Airline D", "10:00 AM", "12:00 PM", "Cancelled"),
    (505, "Airline E", "06:00 AM", "08:00 AM", "On Time")
]

def search_flight(flight_id):
    for flight in flights:
        if flight[0] == flight_id:
            return flight
    return "Flight ID not found"

def sort_flights_by_departure():
    return sorted(flights, key=lambda x: x[2])

def sort_flights_by_arrival():
    return sorted(flights, key=lambda x: x[3])

# Example usage
print(search_flight(503))
print("Flights sorted by departure time:")
for flight in sort_flights_by_departure():
    print(flight)
print("Flights sorted by arrival time:")
for flight in sort_flights_by_arrival():
    print(flight)
```

Output:

```
(503, 'Airline C', '07:30 AM', '09:30 AM', 'On Time')
Flights sorted by departure time:
(505, 'Airline E', '06:00 AM', '08:00 AM', 'On Time')
(503, 'Airline C', '07:30 AM', '09:30 AM', 'On Time')
(501, 'Airline A', '08:00 AM', '10:00 AM', 'On Time')
(502, 'Airline B', '09:00 AM', '11:00 AM', 'Delayed')
(504, 'Airline D', '10:00 AM', '12:00 PM', 'Cancelled')
Flights sorted by arrival time:
(505, 'Airline E', '06:00 AM', '08:00 AM', 'On Time')
(503, 'Airline C', '07:30 AM', '09:30 AM', 'On Time')
(501, 'Airline A', '08:00 AM', '10:00 AM', 'On Time')
(502, 'Airline B', '09:00 AM', '11:00 AM', 'Delayed')
(504, 'Airline D', '10:00 AM', '12:00 PM', 'Cancelled')
```

Explanation: Binary Search can efficiently find flight details by ID.
Sorting by departure or arrival time improves scheduling management.
Efficient algorithms ensure faster operations.
This enhances airport system performance.

